

Project Proposal - Using Linear Regression In A Distributed System To Predict Median House Value

Chris Recupero

4/15/2026

CUS-680 Distributed Big Data Analytics

Dr. Arafat

Team Size: 1

Project Overview

The goal of this project is to predict median house value by census driven districts on the 'California housing Prices' dataset via big distributed data systems and processes. The dataset is a CSV file from kaggle and contains 10 columns total with 20,640 rows; of these 10 columns, 9 are numerical decimal values and 1 is a string. And of these 9 numerical decimal values, 5 are discrete and 4 are continuous. The column we are setting our target as to predict will be "median_house_value," described to be the median value of houses within a block of the house indexed in the row. In this dataset, there is no key to index each individual house by its own ID; this can be something we temporarily create in the dataset in order to do some preliminary exploratory data analysis, by checking if each row is using its own distinct longitude/latitude values. There seem to be some duplicate longitude/latitude pair values in this dataset which will need to be filtered out in order to make sure each row is referencing a unique and non-null house ID. Once it is time to predict the median house value, we will filter this ID column out in our pre-processing since ID values are not features in models.

This is a regression modeling problem. We want to predict the numerical median house value using the set of features given to us in the California Housing Prices dataset. To do so, we will store our data into an HDFS using our Docker container to distribute the storage, processing, and analysis of our data across multiple virtual nodes. We won't have to dig into our local directory anymore to get our data — it's stored in the virtual HDFS. We then use SparkSQL to query our data to do a little exploratory data analysis. We observe distributions of our numerical data; what the interquartile range looks like amongst all of our numerical columns; we find where any NAs are located. This should help us learn a little about our data before we go into the modeling portion so we can understand which features might need pre-processing. Specifically, using the information we get from looking for NA values will lead us right into MapReduce. We use MapReduce to find out how the median house values look keyed and paired by ocean_proximity as a way of exploring how this categorical variable might affect our data. Subsequently, this leads us to Spark. We will use Spark to engineer any features we need to process before putting them into the model based on whatever we found through our EDA in SparkSQL. Additionally, we will split the data in training and testing sets as one last act before we officially make our models. We

made four models to compare evaluations across and then scaled each of these models up on this same dataset multiplied by various factors to see how the scaling would effect the outcome. The four models we used were all linear regression: one with solver = “normal” for non-gradient OLS, another gradient linear regression model with no lasso/regparam, then a lasso linear model and a ridge [model](#). We evaluate the R^2 and rmse across all four models, took the one with the best results, and used that model to scale our data up to see how these same metrics performed on higher volume data. This is all done using SparkMLlib algorithms and preprocessing transformers such as: Imputer, StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler, LinearRegression, Pipeline, and RegressionEvaluator.

The Architecture: The Distributed Model Skeleton

The architecture implemented in this project is based off three core components: the driver (my Macbook pro), two docker clusters (a hadoop container and a spark container), and the hdfs system. Because we are working with two docker clusters, the spark client connects the spark-master and spark-worker to the namenode, datanodes, and nodemanagers. Typically, everything exists in one network and each container can interact freely with others inside the network. Spark-workers could directly read from the namenode and directly take the reduced blocks from the datanodes. In this setup, however, there are two entirely different docker clusters that independently run from each other. Because the spark-worker couldn't get the data from the namenode or datanodes, data needed to flow through a bridge network via the spark-client. The benefits of doing so include: fault isolation in case anything goes wrong in the spark cluster, nothing is affected in the HDFS; independent scalability in both clusters where if I needed to use more datanodes in my HDFS or more worker nodes in my spark cluster, I could scale each cluster independently to my needs (in a project where scalability was a concern, this stuck out to me).

Part 1: Moving Data Into HDFS And Making Our Directories

In terms of how this all worked by phase in this entire pipeline, let's break it down. We first start by taking the housing_2.csv file from our downloads folder and we copy it into the data folder on my MacBook (bigdata/docker-spark/data directory). We then have to move this data from this folder into the docker before it gets moved into the HDFS, which we do in two ways, one for each cluster: for the spark cluster, we do so by

using a volume mount (which is created in our yml file). This volume mount doesn't physically remove the file from our local directory to the docker, rather provides a temporary window to read/see the file (this volume mount is visible through: /opt/data). For the hadoop cluster, we do using docker cp to physically move the file into our docker container containing the HDFS to the docker directory: tmp/. Now we create three more subdirectories using docker exec -i docker-hadoop-namenode-1 hdfs dfs -mkdir -p /user/housing/"name of new subdirectory"...we create three new ones called "raw," "partitioned," and "processed". The partitioned folder will contain the 5 ocean_prox partitioned datasets (near_bay, <1H Ocean, inland, near_ocean, and island). We finally are able to now push the file into the hdfs inside the housing/raw subdirectory.

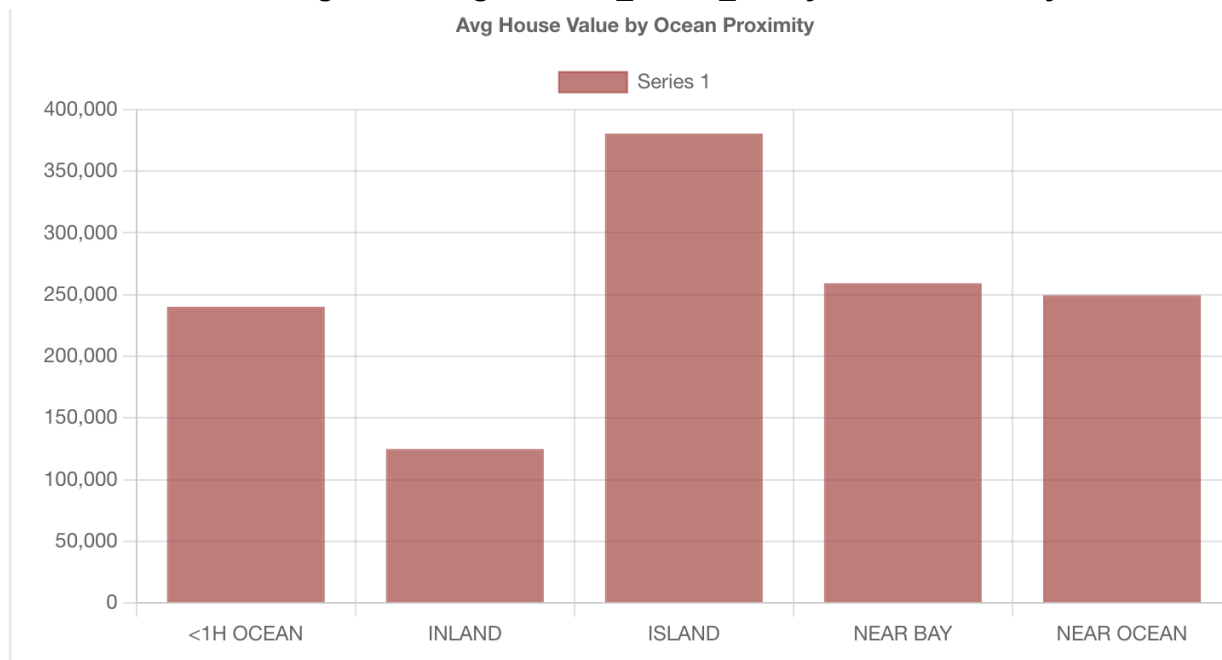
Part 2: MapReduce

Now it's time for MapReduce. MapReduce takes the [mapper.py](#) and [reducer.py](#) scripts from the /tmp/ folder in the docker and uploads them to the HDFS as temporary job resources. Every nodemanager in the cluster then receives these tasks and runs them on deployment; the distribution to the nodemanagers and overall logistics is handled by hadoop accordingly. YARN via the resourcemanager inspects the available resources in the nodemanagers and distributes based on this. In terms of the actual data, CSV is split twice for this specific project; each split is assigned to a separate mapper task on a nodemanager container. The shuffle/sort part of MapReduce then collects all the key-pair outputs from the mappers, sorts it by the 5 keys (the ocean_prox values), and send them all individually to a reducer. Here, the total amount of rows per key and the average median house_value for the districts in these keys was output and wrote into the HDFS at /user/housing/mapreduce_output/part-00000. Here, we saw that:

Fig 2.1 MapReduce Output

Ocean Proximity	Count	Total Value (\$)	Avg House Value (\$)
<1H OCEAN	9,136	\$2,193,410,032.00	\$240,084.29
INLAND	6,551	\$817,600,123.00	\$124,805.39
ISLAND	5	\$1,902,200.00	\$380,440.00
NEAR BAY	2,290	\$593,596,194.00	\$259,212.31
NEAR OCEAN	2,658	\$662,995,512.00	\$249,433.98

Fig 2.2 Average median_house_val By Ocean Proximity



Part 3: SparkSQL

Following the MapReduce phase, we move into our SparkSQL analytics in order to gage a clearer view into our data. We will perform some exploratory data analysis queries on our housing data in order to learn a little more about our dataset before throwing it into the modeling phase. The Spark-client submits the script while it's connected to both networks via the volume mount carrying the scripts. The data is still coming from the HDFS system via the bridge. The driver executes the `spark.read.csv`, it allows the namenode to get the metadata and block locations. Spark-worker then streams the data blocks from the datanodes in the HDFS network into memory, performs whichever aggregations and queries applied in the script, and then sends it back to the driver for printing (which is what we see on the terminal). Everything was written in memory and nothing was saved on disk. We ran five queries in this phase.

The first showed us a count/average/min/max summarization of `house_vals` by `ocean_prox`, similar to what we did in the MapReduce only with more information. We see the same returns we got from MapReduce only we see the minimum house value on the Island is significantly more expensive than any other land while the trends we saw in the averages column carried over to the minimum column, where we saw Inland have the lowest value. Every house block had the same maximum value besides Island. The next query ran average `house_val` calculations and an average income summary by `income_bracket`, where we see that average median_house_value goes up by income group, which is obvious. The income brackets over \$60,000/yr are nearly double that of the medium bracket (\$20,000 - \$40,000). Most of the population in this dataset are from the medium group (10077 housing blocks in this

census). The third query was summarising age by ocean_prox, where we see the closer to the water, the older the population. The fourth query was a filtered aggregate query for the inland income and inland housing_value, which we see at \$124,805.39 and \$30,209 respectively. Lastly, we see aggregated longitudinal/latitudal data for each ocean_prox along with age, income, and median_house_val data. There doesn't seem to be much variance in the longitude and latitude columns, so they look like they won't be very good predictors.

Fig 3.1 Here's how each query's run time performed, all below 2.0 s with a data load (the time it takes to upload the data to the cluster) is under 8.0s.

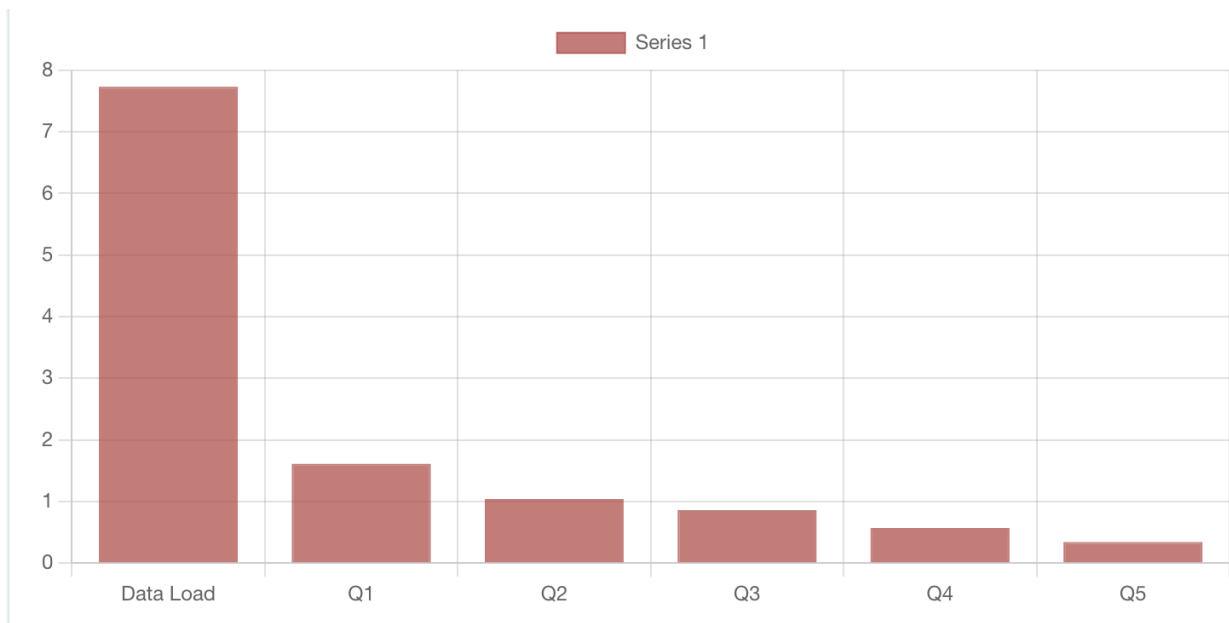


Fig 3.2 Average median_house_val By Ocean Proximity

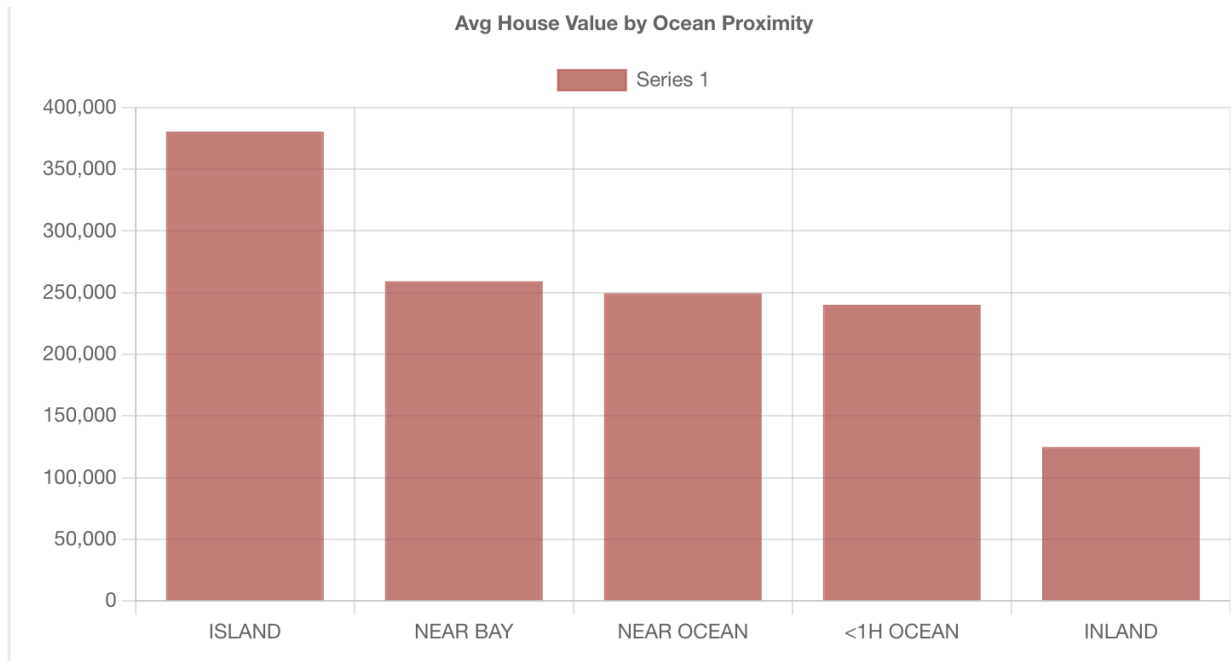


Fig 3.3 Average median_house_val By Income Bracket

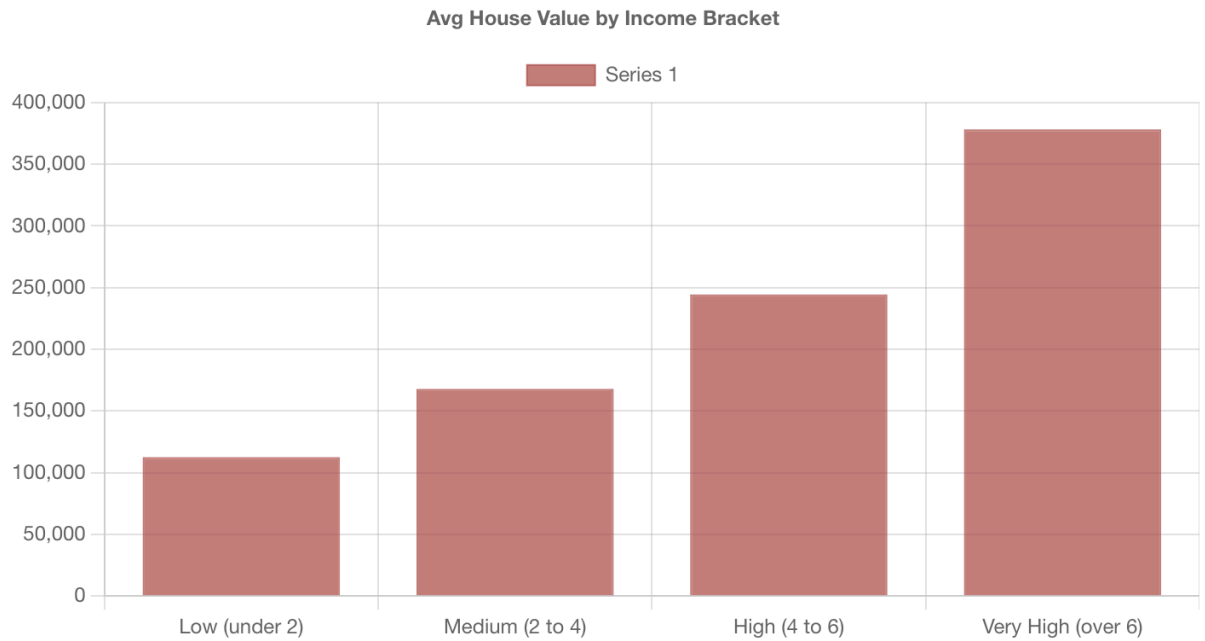
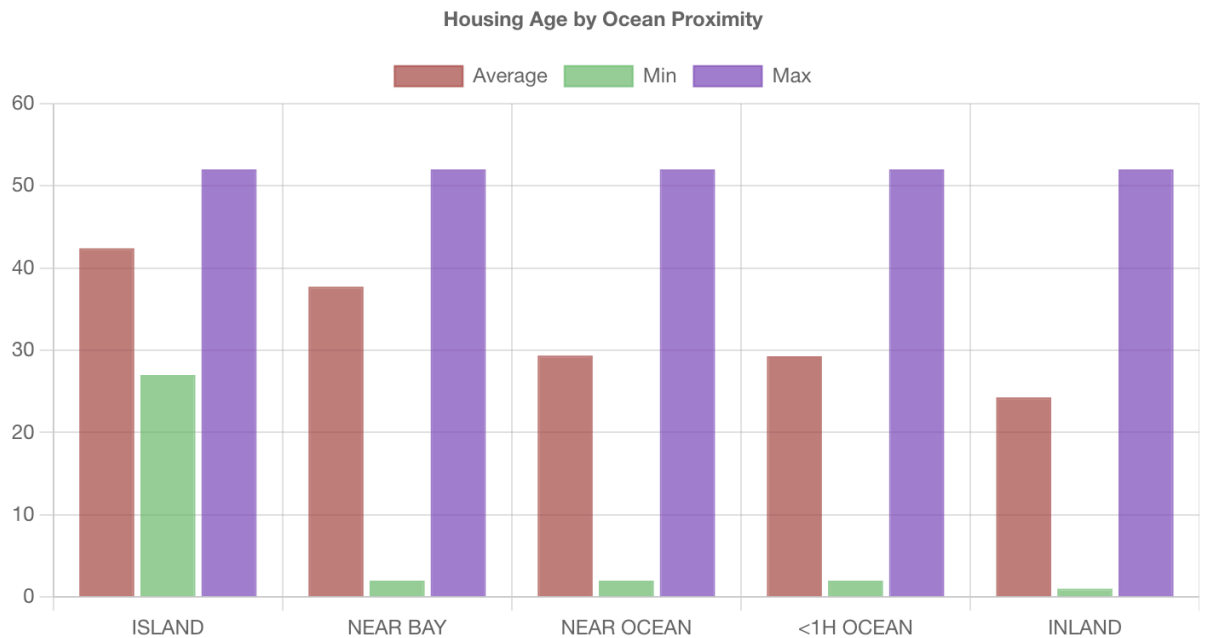


Fig 3.4 Average, Min, and Max Age By Ocean Proximity



Part 4: Spark MLib

Second to lastly is the Spark MLib phase where we take our data, preprocess it a little with some cleaning, standardizations, and transformations, then train it under some linear model frameworks, and compare the evaluation metrics across each model type. The spark-worker reads the training data from the HDFS in partitions and applies the transformations on the data, computes the gradients (on the three models in this project that required them), and sends the results back to the spark-client (the driver in this cluster). The spark-client then combines these results, holds onto the final trained models and their weights, and runs the predictions on the test data using what it holds from the trained models. In this project, we applied the following transformations: imputed missing bedroom values with their medians; converted ocean_prox values to factor dummy vars then one-hot-encoded these values; scaled every numerical feature on a scale of mean 0 with standard deviation 1 (this prevents any one numerical feature from dominating the others). We trained: a normal OLS linear reg model, a non-lasso/penalty implemented gradient descent model iterated on 100 loops; a lasso and ridge linear reg model. All of which got combined to find the final RMSE and R^2 of each. We see the lasso regression model moderately outperformed the other three in RMSE while all four had the same R^2 . The RMSE values imply the model is not where we'd want it to be: the errors were in the \$60,000+ which is quite high. The R^2 implies the model did well in understanding and displaying the overall relationship/trend between the features and median_house_val of each district.

Fig 4.1 Linear Regression Model Evaluation Performances

Model	RMSE	R^2	Train Time (s)
Normal Equation	67,539.15	0.6506	5.12
Gradient Descent	67,539.14	0.6506	8.05
Ridge (L2)	67,539.16	0.6506	4.55
Lasso (L1)	67,538.85	0.6506	9.38

Part 5: Scalability Analysis

This moves us to our final point in the project where we pick our best performing model (Lasso) and scale the data up by looped multipliers to understand how the loss function and R^2 might be affected by the data size. The spark-client runs a loop 5 times over to union together progressively larger data frames. The CSV from the HDFS is called through the loops when the preprocessing calls on the data to be split up, processed, and trained. The results yielded point out that the scale at 20x the data size is the best performing set, giving us an RMSE of \$68,000 with an R^2 of .647 (explaining about 65% of the variance in the data). The training time was obviously the longest, but at only 30.0s of training time for a > \$2,000 difference in RMSE, the benefits outweighed the costs of scaling the data up.

Fig 5.1 Scalability Evaluation On Lasso Regression Model

Data Mult	Rows	T Elapsed (s)	RMSE	R ²
1x	20,640	21.16	70,802.34	0.6377
2x	41,280	8.87	69,559.03	0.6440
5x	103,200	16.49	69,122.95	0.6430
10x	206,400	19.68	68,520.47	0.6489
20x	412,800	29.77	68,800.14	0.6467

Conclusion

Ultimately, we can see from the chart (as stated above) the lasso regression model performed the best out of the four model predictors and when scaled up across the 5 multipliers, performed best at 20x the volume as per the RMSE and R². Ideally, in a world where I am trying to reduce the loss function and maximize the variance explained by my features via my machine learning phase as much as possible, I'd implement a tuning phase on 5-fold cross-validated training data and optimize my features/parameters as much as possible. Going forward, I would get rid of longitude/latitude from my feature set, undergo feature reduction by either using one of either totalRoom and totalBedrooms, or engineer a feature that blends both of the information together.